

University
Politehnica
of Bucharest





PostgreSQL – PL/pgSQL

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro



Overview

- Indexing
- Procedural Language



Overview

- Indexing
- Procedural Language



PostgreSQL Indexing

- PostgreSQL provides several index types: B-tree, Hash, GiST, SP-GiST, GIN and BRIN

- Each index type uses a different algorithm that is best suited to different types of queries

- B-trees
  - Can handle equality and range queries on data that can be sorted into some ordering
  - The PostgreSQL query planner will consider using a B-tree index whenever an indexed column is involved in a comparison
  - B-tree indexes can also be used to retrieve data in sorted order:
    - Not always faster than a simple scan and sort, but it is often helpful

PostgreSQL Indexing

- Hash indexes
  - Can only handle simple equality comparisons
  - The query planner will consider using a hash index whenever an indexed column is involved in a comparison using the = operator.

- GiST indexes
  - Are not a single kind of index, but rather an infrastructure within which many different indexing strategies can be implemented
  - The particular operators with which a GiST index can be used vary depending on the indexing strategy (the *operator class*)
  - The standard distribution of PostgreSQL includes GiST operator classes for several two-dimensional geometric data types
  - GiST indexes are also capable of optimizing "nearest-neighbor" searches

PostgreSQL Indexing

- SP-GiST indexes
  - Offer an infrastructure that supports various kinds of searches.
  - Permits implementation of a wide range of different non-balanced disk-based data structures, such as quadtrees, k-d trees, and radix trees
  - Supports "nearest-neighbor" searches

- GIN indexes
  - Are "inverted indexes" which are appropriate for data values that contain multiple component values, such as arrays
  - An inverted index contains a separate entry for each component value and can efficiently handle queries that test for the presence of specific component values
  - Can support many different user-defined indexing strategies, and the particular operators with which a GIN index can be used vary depending on the indexing strategy



PostgreSQL Indexing

- BRIN indexes (a shorthand for Block Range INdexes)
  - Store summaries about the values stored in consecutive physical block ranges of a table
  - Can support many different indexing strategies
  - The operators with which a BRIN index can be used vary depending on the indexing strategy
  - The indexed data corresponds to the minimum and maximum values of the values in the column for each block range (for data types that have a linear sort order)



PostgreSQL Multicolumn Indexes

- An index can be defined on more than one column of a table
- Only the B-tree, GiST, GIN, and BRIN index types support multicolumn indexes (up to 32 columns can be specified)
- A multicolumn index can be used with query conditions that involve any subset of the index's columns:
  - B-tree index is most efficient when there are constraints on the leading (leftmost) columns
  - GiST index is most efficient when the condition on the first column is the most important one (determines how much of the index needs to be scanned)
  - GIN and BRIN indexes effectiveness is the same regardless of which index column(s) the query conditions use



PostgreSQL Multicolumn Indexes

- Multicolumn indexes should be used sparingly
- In most situations, an index on a single column is
  - Sufficient
  - Saves space and time
- Indexes with more than three columns are unlikely to be helpful unless the usage of the table is extremely stylized



PostgreSQL Indexes

```sql
CREATE TABLE test (
    id INTEGER,
    title TEXT,
    content TEXT,
    loc point,
    date TIMESTAMP DEFAULT current_timestamp
);
-- b-tree
CREATE INDEX idx_id ON test(id);
DROP INDEX idx_id;
-- b-tree
CREATE INDEX idx_id ON test USING BTREE (id);
DROP INDEX idx_id;
-- hash
CREATE INDEX idx_id ON test USING HASH (id);
DROP INDEX idx_id;
-- gist
CREATE INDEX idx_loc ON test USING GIST (loc);
DROP INDEX idx_loc;
-- spgist
CREATE INDEX idx_loc ON test USING SPGIST (loc);
DROP INDEX idx_loc;
-- gin
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_text ON test USING GIN (title gin_trgm_ops, content gin_trgm_ops);
DROP INDEX idx_text;
-- brin
CREATE INDEX idx_date ON test USING BRIN (date);
DROP INDEX idx_date;
```



Overview

- Indexing
- Procedural Language



PostgreSQL Procedural Language

- PostgreSQL supports multiple procedural languages (PLs)

- For a function written in a procedural language
  - The database server has no built-in knowledge about how to interpret the function's source text
  - The task is passed to a special handler that knows the details of the language
- The handler could
  - Do all the work of parsing, syntax analysis, execution, etc.
  - Serve as "glue" between PostgreSQL and an existing implementation of a programming language
- The handler itself is a C language function compiled into a shared object and loaded on demand

- There are currently four procedural languages available in the standard PostgreSQL distribution: **PL/pgSQL**, PL/Tcl , PL/Perl, PL/Python



PostgreSQL
PL/pgSQL

- PL/pgSQL is a loadable procedural language for the PostgreSQL database system

- The design goals of PL/pgSQL were to create a loadable procedural language that
  - Can be used to create functions and triggers
  - Adds control structures to the SQL language
  - Can perform complex computations
  - Inherits all user-defined types, functions, and operators
  - Can be defined to be trusted by the server
  - Is easy to use

- Functions created with PL/pgSQL can be used anywhere that built-in functions could be used



PostgreSQL
PL/pgSQL Advantages

- SQL is the language PostgreSQL and most other relational databases use as query language
- Downside, SQL statement must be executed individually by the database server
- That means that your client application must
  - Send each query to the database server
  - Wait for it to be processed
  - Receive and process the results
  - Do some computation
  - Send further queries to the server
- All this incurs
  - Inter-process communication
  - Network overhead



PostgreSQL
PL/pgSQL Advantages

- With PL/pgSQL a developer can
  - Group a block of computation and a series of queries inside the database server
  - Combine the functionalities of a procedural language with SQL
  - Minimize considerable client/server communication overhead
  - Use all the data types, operators and functions of SQL

- PL/pgSQL eliminates
  - Extra round trips between client and server
  - Intermediate results
    - That the client does not need do
    - That do not need to be transferred between server and client
  - Multiple rounds of query parsing can be avoided

- This can result in a considerable performance increase as compared to an application that does not use stored functions



PostgreSQL
PL/pgSQL Functions

- Functions written in PL/pgSQL can
  - Accept as arguments scalar or array data type supported by the server
  - Return a result of any of these types
  - Accept or return any composite type (row type) specified by name

- It is also possible to declare a PL/pgSQL function that
  - Accepts a record parameter (any composite type will do as input)
  - Returns a record (the result is a row type whose columns are determined by specification in the calling query)



PostgreSQL
PL/pgSQL Functions

- PL/pgSQL functions can be declared
  - To accept
    - A variable number of arguments by using the VARIADIC marker
    - Polymorphic types:
      - Anyelement
      - Anyarray
      - Anynonarray
      - Anyenum
      - Anyrange
  - To return
    - A "set" (or table) of any data type that can be returned as a single instance
    - **void** if it has no useful return value



PostgreSQL
PL/pgSQL Functions

- Functions written in PL/pgSQL are defined to the server by executing **CREATE FUNCTION** command

- Syntax:

CREATE FUNCTION somefunc(integer, text) RETURNS integer
AS
'***function body text***'
LANGUAGE plpgsql;

- The function body is simply a string literal so far as CREATE FUNCTION is concerned

- It is often helpful to use dollar quoting to write the function body, rather than the normal single quote syntax.



PostgreSQL
PL/pgSQL Functions

- PL/pgSQL is a block-structured language. The complete text of a function body must be a *block*.

[<<***label***>>]
[DECLARE
  declarations;]
BEGIN
  statements;
[EXCEPTION
  WHEN ... THEN statements;
  WHEN ... THEN statements;
...]
END [***label***]

- Each declaration and each statement within a block is terminated by a semicolon
- A block that appears within another block must have a semicolon after END
- The final END that concludes a function body does not require a semicolon



PostgreSQL
PL/pgSQL Functions

```
 1 CREATE OR REPLACE FUNCTION somefunc() RETURNS integer AS $$
 2 <<outerblock>>
 3 DECLARE
 4     quantity integer := 30;
 5 BEGIN
 6     RAISE NOTICE 'Quantity here is %', quantity;
 7     quantity := 50;
 8     -- CREATE SUBBLOCK
 9     DECLARE
10         quantity integer := 80;
11     BEGIN
12         RAISE NOTICE 'Quantity here is %', quantity;
13         RAISE NOTICE 'Outer quantity here is %', outerblock.quantity;
14     END;
15     RAISE NOTICE 'Quantity here is %', quantity;
16     RETURN quantity;
17 END;
18 $$ LANGUAGE plpgsql;
19
20 SELECT somefunc;
```



PostgreSQL
PL/pgSQL Functions Declarations

- All variables used in a block must be declared in the declarations section of the block

- PL/pgSQL variables can have any SQL data type

name [CONSTANT] ***type*** [COLLATE ***collation_name***] [NOT NULL] [{ DEFAULT | := | = } ***expression***];

- Parameters passed to functions are named with the identifiers $1, $2, etc.

- Optionally, aliases can be declared for $***n*** parameter names for increased readability

- Either the alias or the numeric identifier can then be used to refer to the parameter value.



PostgreSQL
PL/pgSQL Functions

```
 1 CREATE FUNCTION sales_tax(subtotal real) RETURNS real AS $$
 2 BEGIN
 3     RETURN subtotal * 0.06;
 4 END;
 5 $$ LANGUAGE plpgsql;
 6
 7 SELECT sales_tax(100);
 8 DROP FUNCTION sales_tax;
 9
10 CREATE FUNCTION sales_tax(real) RETURNS real AS $$
11 BEGIN
12     RETURN $1 * 0.06;
13 END;
14 $$ LANGUAGE plpgsql;
15
16 SELECT sales_tax(100);
17 DROP FUNCTION sales_tax;
18
19 CREATE FUNCTION sales_tax(real) RETURNS real AS $$
20 DECLARE
21     subtotal ALIAS for $1;
22 BEGIN
23     RETURN subtotal * 0.06;
24 END;
25 $$ LANGUAGE plpgsql;
26
27 SELECT sales_tax(100);
28 DROP FUNCTION sales_tax;
```



PostgreSQL
PL/pgSQL Functions

- When a PL/pgSQL function is declared with output parameters
  - The output parameters are given $***n*** names
  - Can have optional aliases (as the normal input parameters)

- An output parameter
  - Is effectively a variable that starts out NULL
  - It should be assigned a value during the execution of the function
  - The final value of the parameter is what is returned

- Parameter modes: IN, OUT, and INOUT

```
1 CREATE FUNCTION sum_n_product(x int, y int, OUT sum int, OUT prod int) AS $$
2 BEGIN
3     sum := x + y;
4     prod := x * y;
5 END;
6 $$ LANGUAGE plpgsql;
7
8 SELECT * FROM sum_n_product(3,4);
```



PostgreSQL
PL/pgSQL Functions

- This effectively creates an anonymous record type for the function's results.
- If a RETURNS clause is given, it must say RETURNS RECORD.

```
1  CREATE FUNCTION sumprod(x int, y int)
2  RETURNS RECORD
3  AS $$
4  DECLARE
5      r RECORD;
6  BEGIN
7      SELECT x + y, x * y INTO r;
8      return r;
9  END;
10 $$ LANGUAGE plpgsql;
11
12 SELECT sumprod(3,4);
```



PostgreSQL
PL/pgSQL Functions

```
 1 CREATE FUNCTION sumprod(
 2   sum INOUT int,
 3   prod INOUT int)
 4 AS $$
 5 DECLARE
 6     s INT;
 7     p INT;
 8 BEGIN
 9     s := sum + prod;
10     p := sum * prod;
11     sum := s;
12     prod := p;
13 END;
14 $$ LANGUAGE plpgsql;
15
16 SELECT * FROM sumprod(3,4);
```



PostgreSQL
PL/pgSQL Functions

- If the function is defined to return a base type, the table function produces a one-column table

- If the function is defined to return a composite type, the table function produces a column for each attribute of the composite type.

```
1 CREATE TABLE friends (friend_id int, name text);
2 INSERT INTO friends VALUES (1, 'Joe');
3 INSERT INTO friends VALUES (2, 'Ed');
4 INSERT INTO friends VALUES (3, 'Mary');
5
6 CREATE FUNCTION getFriend(int) RETURNS friends AS
7 $$
8     SELECT * FROM friends WHERE friend_id = $1;
9 $$
10 LANGUAGE SQL;
11
12 SELECT name FROM getFriend(1);
```



PostgreSQL
PL/pgSQL Functions

```
 1 CREATE FUNCTION getFriend(fid int)
 2 RETURNS TABLE(id int, n text) AS $$
 3 BEGIN
 4     RETURN QUERY SELECT friend_id, name FROM friends
 5                  WHERE friend_id = fid;
 6 END;
 7 $$ LANGUAGE plpgsql;
 8
 9 SELECT getFriend(1);
10
11 SELECT * FROM getFriend(1);
```



PostgreSQL
PL/pgSQL Functions

- A function can return a polymorphic type (anyelement, anyarray, anynonarray, anyenum, or anyrange)

Type	Description
anyelement	Indicates that a function accepts any data type
anyarray	Indicates that a function accepts any array data type
anynonarray	Indicates that a function accepts any non-array data type
anyenum	Indicates that a function accepts any enum data type
anyrange	Indicates that a function accepts any range data type

- A special parameter $0 is created when the return type of a PL/pgSQL function is declared as a polymorphic type
- $0 is initialized to null and can be modified by the function, so it can be used to hold the return value if desired, though that is not required



PostgreSQL
PL/pgSQL Functions

```
 1 CREATE FUNCTION add3vals(
 2   v1 anyelement,
 3   v2 anyelement,
 4   v3 anyelement)
 5 RETURNS anyelement
 6 AS $$
 7 DECLARE
 8     result ALIAS for $0;
 9 BEGIN
10     result := v1 + v2 + v3;
11     RETURN result;
12 END;
13 $$ LANGUAGE plpgsql;
14
15 SELECT add3vals(1,2,3);
```



PostgreSQL
PL/pgSQL Functions

- %TYPE
  - Provides the data type of a variable or table column
  - Useful to declare variables that will hold database values

- %ROWTYPE
  - A variable of a composite type is called a row variable (or row-type variable)
  - Such a variable can hold a whole row of a SELECT or FOR query result



PostgreSQL
PL/pgSQL Functions

```
 1 CREATE OR REPLACE FUNCTION getInfo(emp_row employees)
 2 RETURNS text
 3 AS $$
 4 DECLARE
 5     dept_row departments%ROWTYPE;
 6     city_name cities.city%TYPE;
 7 BEGIN
 8     SELECT * INTO dept_row
 9     FROM departments
10     WHERE department_id = emp_row.department_id;
11
12     SELECT c.city INTO city_name
13     FROM cities c
14     INNER JOIN locations l ON c.city_id = l.city_id
15     INNER JOIN departments d ON l.location_id = d.location_id
16     WHERE d.department_id = dept_row.department_id;
17
18     RETURN emp_row.first_name ||' '||
19             emp_row.last_name ||' '||
20             dept_row.department_name ||' '||
21             city_name;
22 END;
23 $$ LANGUAGE plpgsql;
24
25 SELECT getInfo(e.*) FROM employees e
26 WHERE e.employee_id =  100;
```



PostgreSQL
PL/pgSQL Control Statements

- Conditionals:
  - IF and CASE statements let you execute alternative commands based on certain conditions.
  - Three forms of IF:
    - IF ... THEN ... END IF
    - IF ... THEN ... ELSE ... END IF
    - IF ... THEN ... ELSIF ... THEN ... ELSE ... END IF
  - Two forms of CASE (Simple and Searched CASE):
    - CASE ... WHEN ... THEN ... ELSE ... END CASE
    - CASE WHEN ... THEN ... ELSE ... END CASE

- Loops:
  - LOOP
  - WHILE
  - FOR
  - FOREACH



PostgreSQL
PL/pgSQL Control Statements

```
 1 CREATE OR REPLACE FUNCTION getEmps()
 2 RETURNS VOID
 3 AS $$
 4 DECLARE
 5     emp RECORD;
 6 BEGIN
 7     FOR emp IN
 8         SELECT first_name, last_name
 9         FROM employees
10     LOOP
11         RAISE NOTICE 'Employee % %',
12             emp.first_name,
13             emp.last_name;
14     END LOOP;
15 END;
16 $$ LANGUAGE plpgsql;
17
18 SELECT getEmps();
```



PostgreSQL
PL/pgSQL Control Statements

```
 1 CREATE FUNCTION sum_arr(int[])
 2 RETURNS int8
 3 AS $$
 4 DECLARE
 5   s int8 := 0;
 6   x int;
 7 BEGIN
 8   FOREACH x IN ARRAY $1
 9   LOOP
10     s := s + x;
11   END LOOP;
12   RETURN s;
13 END;
14 $$ LANGUAGE plpgsql;
15
16 SELECT sum_arr('{2, 3, 5, 7}'::int[4]);
```



PostgreSQL
PL/pgSQL Cursors

- Rather than executing a whole query at once, it is possible to
  - Set up a cursor that encapsulates the query
  - Read the query result a few rows at a time

- One reason for doing this is to avoid memory overrun when the result contains many rows

- Declaration

  name [[NO] SCROLL] CURSOR [(***arguments***)] FOR ***query***;

- To declare a cursor variable use: **refcursor**



PostgreSQL
PL/pgSQL Cursors

- To open a cursor use OPEN
  OPEN unbound_cursor_variable [[NO] SCROLL] FOR query;

- **SCROLL** option is used to speciffy Whether the cursor can be scrolled backward or not

- To fetch the next row use

FETCH [direction { FROM | IN }] cursor_variable INTO target_variable;

- By default, a cursor gets the next row if no direction is specified explicitly

- The following are valid cursor directions:
  - NEXT, LAST, PRIOR, FIRST
  - ABSOLUTE count, RELATIVE count
  - FORWARD, BACKWARD (only for cursors declared with SCROLL option)



PostgreSQL
PL/pgSQL Cursors

- To move the cursor without retrieving any rows

  MOVE [direction { FROM | IN }] cursor_variable;

- The direction accepts the same value as the **FETCH** statement.

- Once a cursor is positioned, developers can update or delete rows identifying by the cursor using **CURRENT OF**

- To close a cursor:

  CLOSE cursor_variable;



PostgreSQL
PL/pgSQL Cursors

```
 1 CREATE OR REPLACE FUNCTION getEmps()
 2 RETURNS VOID
 3 AS $$
 4 DECLARE
 5   emp RECORD;
 6   depts CURSOR (id integer) FOR
 7   SELECT department_name FROM departments
 8   WHERE department_id = id;
 9   dept_name departments.department_name%TYPE;
10 BEGIN
11   FOR emp IN
12     SELECT first_name, last_name, department_id
13     FROM employees
14   LOOP
15     OPEN depts(emp.department_id);
16     LOOP
17       FETCH depts INTO dept_name;
18       EXIT WHEN NOT FOUND;
19       RAISE NOTICE 'Employee % % works in department %',
20           emp.first_name, emp.last_name, dept_name;
21     END LOOP;
22     CLOSE depts;
23   END LOOP;
24 END;
25 $$ LANGUAGE plpgsql;
26
27 SELECT getEmps();
```



PostgreSQL
PL/pgSQL Trigger Functions

- PL/pgSQL can be used to define trigger functions on data changes or database events

- A trigger function is created with the CREATE FUNCTION command
  - Declaring it as a function with no arguments
  - A return type of
    - trigger (for data change triggers)
    - event_trigger (for database event triggers).



PostgreSQL
PL/pgSQL Trigger Functions

```
CREATE OR REPLACE FUNCTION add_comm() RETURNS trigger AS $add_comm$
  BEGIN
    IF COALESCE(OLD.commission_pct, 0) < COALESCE(NEW.commission_pct, 0) THEN
      RAISE NOTICE 'commission_pct increased';
    ELSIF COALESCE(OLD.commission_pct, 0) > COALESCE(NEW.commission_pct, 0) THEN
      RAISE NOTICE 'commission_pct decreased';
    ELSE
      RAISE NOTICE 'remained the same';
    END IF;
    RETURN NULL;
  END;
$add_comm$ LANGUAGE plpgsql;

CREATE TRIGGER add_comm BEFORE INSERT OR UPDATE ON employees
    FOR EACH ROW EXECUTE FUNCTION add_comm();


INSERT INTO employees VALUES(501, 'Ion', 'Popescu', 'ion.popescu@corp.com',
                             '515.123.4567',current_date, 'PU_CLERK', 3100.00, NULL, 100, 90);
INSERT INTO employees VALUES(502, 'Jon', 'Popescu', 'ion.popescu@corp.com',
                             '515.123.4567',current_date, 'PU_CLERK', 3100.00, 0.5, 100, 90);
UPDATE employees SET commission_pct = 0.3  WHERE employee_id=501;
UPDATE employees SET commission_pct = 0.3  WHERE employee_id=502;
```